

# 0.1 - PATTERN REPRESENTATION

## CHAPTER INDEX

- ☐ [DATA AND PATTERNS](#)
  - [KINDS OF PATTERNS](#)
- ☐ [PATTERN REPRESENTATION IN BIOLOGY](#)
  - [PROTEINS](#)
  - [DNA](#)
- ☐ [OTHER APPROACHES](#)
  - [LOCAL BINARY PATTERNS](#)
  - [DISSIMILARITY SPACES](#)

## DATA AND PATTERNS

Data is a **fundamental ingredient of machine learning**, where the behaviour of the algorithms is *not pre-programmed*, but **learned from the data** itself.

Terms like Data Science, Data Mining, Big Data emphasise the role of data.

We will often use the term "**pattern**" to refer to data.

### PATTERN (Def. by S. Watanabe)

A *pattern* can be defined as **the opposite of chaos**.

| A *vaguely defined entity that can be given a name*.

For example, a pattern can be a *face*, a *handwritten character*, a *fingerprint*, a *sound signal*, a *fragment of text*, the *performance of a stock exchange*, and so on.

### PATTERN RECOGNITION

The discipline that *deals with the recognition of patterns*.

**NOTE:** Not necessarily with ML, it may also use *pre-programmed algorithms*.

## KINDS OF PATTERNS

There are **many kinds** of patterns, let's take a look at some of them.

### NUMERICAL PATTERNS

Values associated with *measurable characteristics*.

- They may be *continuous* or *discrete*, in both cases *subject to order*.

- They are naturally **represented** as **numerical vectors** in *multidimensional space*.

### ☰ EXAMPLE OF NUMERICAL PATTERN

Person: [height, chest circumference, hip circumference, foot length]

### 📖 CATEGORICAL PATTERNS

Values associated with *qualitative characteristics* and the *presence/absence* of a feature (*yes/no* value).

- They are not *semantically* mappable into numerical values.
- They are normally managed by *rule systems* and *classification trees*.

### 📖 SEQUENTIAL PATTERNS

Sequences with *spatial* or *temporal relationships*, often *variable in length*.

- The *position in the sequence* and *relationships with predecessors and successors* are fundamental.
- They can be thought of as numerical patterns where *components are spatially or temporally related*.

### ☰ EXAMPLES OF SEQUENTIAL PATTERNS

Audio streams, text streams, proteins (sequences of amino acids).

## PATTERN REPRESENTATION IN BIOLOGY

As seen above, *proteins* are examples of *sequential patterns*, and so are *DNA fragments*. As a consequence, *pattern representation* finds many applications in *bioinformatics*, *drug development* and, in general, in all of *biology*.

Here, we present a few examples of such applications with the intent of building an *intuition* for developing pattern representation *strategies* in future cases.

## PROTEINS

### 💡 REMINDER

A **protein** is a *sequence of amino acids*. There exist 20 different amino acids in nature.

|     |   |                 |     |   |              |
|-----|---|-----------------|-----|---|--------------|
| Ala | A | Alanina         | Leu | L | Leucina      |
| Arg | R | Arginina        | Lys | K | Lisina       |
| Asn | N | Asparagina      | Met | M | Metionina    |
| Asp | D | Acido aspartico | Phe | F | Fenilalanina |
| Cis | C | Cisteina        | Pro | P | Prolina      |
| Gly | G | Glicina         | Ser | S | Serina       |
| Glu | E | Acido glutamico | Thr | T | Treonina     |
| Gln | Q | Glutamina       | Try | W | Triptofano   |
| His | H | Istidina        | Tyr | Y | Tirosina     |
| Ile | I | Isoleucina      | Val | V | Valina       |

A naive way to represent a protein of length  $N$  would be using a vector  $AS \in \mathbb{R}^N$ , each component is given by:

$$AS(i) = \frac{h(i)}{N}, i \in [1, \dots, 20]$$

Here,  $h(i)$  is the *number of occurrences* of a given amino acid in the protein.

This solution is an example of **purely numerical** pattern, but we can do better: we know that the **position** of an amino acid in the protein is not random, it *carries a meaning* that we aren't *encoding* with the representation above.

We thus want to find a representation strategy that takes this extra information into account.

A first step in this direction could be assuming that one amino acid's *role* could be influenced by the *presence of another*: this means **counting the occurrences of each pair** of amino acids.

This representation is known as **2-Gram (2G)** and is defined as follows:

$$2G(k) = \left( \frac{h(i, j)}{N} \right), i, j \in [1, \dots, 20]$$

Here,  $k = j + 20(i - 1)$  iterates over *all possible pairs*.

A more complex approach is given the **autocovariance (AC) representation**.

Given a parameter  $m$  denoting the *maximum distance between two different amino acids* in a protein  $P = (p_1, p_2, \dots, p_N)$  of length  $N$ , and a *physicochemical property*  $d$ ,  $AC^d(i)$  is defined as follows:

$$AC^d(i) = \begin{cases} h(i)/N & i \in [1, \dots, 20] \\ \sum_{k=1}^{N-i+20} \frac{(\text{index}(p_k, d) - \mu_d)(\text{index}(p_{k+i-20}) - \mu_d)}{\sigma_d(N-i+20)} & i \in [21, \dots, 20+m] \end{cases}$$

Therefore  $AC^d \in \mathbb{R}^{20+m}$ .

**NOTE:** The parameter  $\mu_d$  is the *mean value* for property  $d$  and  $\sigma_d$  is the *variance* of the values for property  $d$ .

#### **OBSERVATION**

The first 20 components of the *feature vector* are simply the *frequencies of each amino acid*.

To better understand the last  $m$  components introduced by this strategy, let's take a look at the following table:

| Amino Acid | Polarity | Secondary structure | Molecular volume | Codon diversity | Electrostatic charge |
|------------|----------|---------------------|------------------|-----------------|----------------------|
| A          | -0.591   | -1.302              | -0.733           | 1.57            | -0.146               |
| C          | -1.343   | 0.465               | -0.862           | -1.02           | -0.255               |
| D          | 1.05     | 0.302               | -3.656           | -0.259          | -3.242               |
| E          | 1.357    | -1.453              | 1.477            | 0.113           | -0.837               |
| F          | -1.006   | -0.59               | 1.891            | -0.397          | 0.412                |
| G          | -0.384   | 1.652               | 1.33             | 1.045           | 2.064                |
| H          | 0.336    | -0.417              | -1.673           | -1.474          | -0.078               |
| I          | -1.239   | -0.547              | 2.131            | 0.393           | 0.816                |
| K          | 1.831    | -0.561              | 0.533            | -0.277          | 1.648                |
| L          | -1.019   | -0.987              | -1.505           | 1.266           | -0.912               |
| M          | -0.663   | -1.524              | 2.219            | -1.005          | 1.212                |
| N          | 0.945    | 0.828               | 1.299            | -0.169          | 0.933                |
| P          | 0.189    | 2.081               | -1.628           | 0.421           | -1.392               |
| Q          | 0.931    | -0.179              | -3.005           | -0.503          | -1.853               |
| R          | 1.538    | -0.055              | 1.502            | 0.44            | 2.897                |
| S          | -0.228   | 1.399               | -4.76            | 0.67            | -2.647               |
| T          | -0.032   | 0.326               | 2.213            | 0.908           | 1.313                |
| V          | -1.337   | -0.279              | -0.544           | 1.242           | -1.262               |
| W          | -0.595   | 0.009               | 0.672            | -2.128          | -0.184               |
| Y          | 0.26     | 0.83                | 3.097            | -0.838          | 1.512                |

In the above example we have 5 *properties* (polarity, secondary structure, ...), therefore  $d = 1, \dots, 5$ .

### EXAMPLE EXPLANATION

In this setting, we would have:

- $\text{index}(A, 1) = -0.591$
- $\text{index}(C, 3) = 0.465$

Now let's define  $\text{lag} := i - 20$ ; with  $i \in [21, \dots, 20 + m]$  we have  $\text{lag} \in [1, \dots, m]$ .

Looking at the definition of the last  $m$  components, we immediately notice that we are checking the **values** (or, more precisely, the distance from the mean) of property  $d$  at **positions**  $k$  and  $k + \text{lag}$ .

### INTERPRETATION

We are considering **pairs** of amino acids that are **separated by lag other amino acids** and iterating for values of lag ranging from 1 (*minimum distance*) to  $m$  (*maximum distance*).

With this approach, we can detect *whether certain properties are influenced by other amino acids at set distances*.

This is important because these relations may emerge because of *interactions* caused by the **protein's 3D structure** (proteins aren't straight, they are folded in complicated ways).

## DNA

The *frequencies of occurrence* can also be used for DNA sequences.

Let's consider a sequence of nucleotides Nuc:

$$\text{Nuc} = [R_l], l = 1, 2, 3, \dots, L$$

We can use both the *occurrence of the single nitrogenous base* (denoted  $f(\text{NitBase})$ ), in which case our *feature vector* would be:

$$F = [f(A), f(C), f(G), f(T)]$$

or the *occurrences of pairs* (denoted  $f(\text{NitBase}_1\text{NitBase}_2)$ ), in which case we would obtain:

$$F = [f(AA), f(AC), f(AG), f(AT), \dots, f(TT)]$$

A more complex approach is given by the *pseudo nucleic acid representation*, which describes a DNA sequence using the following  $g_i$  entries for the feature vector:

$$g_1 = \frac{1}{L-2} \sum_{i=1}^{L-2} \Delta(R_i R_{i+1}, R_{i+1} R_{i+2})$$

$$g_2 = \frac{1}{L-3} \sum_{i=1}^{L-3} \Delta(R_i R_{i+1}, R_{i+2} R_{i+3})$$

$$g_1 = \frac{1}{L-4} \sum_{i=1}^{L-4} \Delta(R_i R_{i+1}, R_{i+3} R_{i+4})$$

...

$$g_\omega = \frac{1}{L-\omega-1} \sum_{i=1}^{L-\omega-1} \Delta(R_i R_{i+1}, R_{i+\omega} R_{i+\omega+1})$$

Where the terms in the summations are given by:

$$\Delta(R_i R_{i+1}, R_k R_{k+1}) = \frac{1}{J} \sum_{j=1}^J [V_j(R_i R_{i+1}) - V_j(R_k R_{k+1})]^2$$

The parameter  $J$  is the number of *properties* and  $V_j(a, b)$  is the value of the  $j$ -th property for the base pair  $ab$  (these values are gathered in tables, just like in the earlier example regarding proteins).

## DNA MICROARRAYS

A DNA microarray (also commonly known as *DNA chip* or *biochip*) is a **collection of microscopic DNA spots** attached to a solid surface.

Scientists use DNA microarrays to *measure the expression levels of large numbers of genes simultaneously* or to genotype multiple regions of a genome.

Each DNA spot contains picomoles (10–12 moles) of a specific DNA sequence, known as a **probe** (or **reporter** or **oligo**).

A typical use is to *compare the gene expression* profile of a *sick individual* with that of a *healthy one* to identify *which genes are involved in the disease*, or to choose the appropriate drug based on gene expression.

The problem is that a microarray allows the expression of over 20,000 genes to be extracted, therefore each DNA sample is described by over 20,000 elements, **most of which are not related to the given classification process**.

Techniques called **features selection** are used to *select only some gene expressions*. E.g. *signal to noise* methods calculate, for each gene  $j$  (example valid for a bi-classification problem: relevant/irrelevant), a coefficient given by:

$$S(j) = \frac{\mu_+(j) - \mu_-(j)}{\sigma_+(j) - \sigma_-(j)}$$

### 💡 APPROACH EXPLANATION

The genes that give positive values are correlated with the class  $+1$ , while the genes that give negative values are correlated with the class  $-1$ .  
At the end,  $m/2$  positive and  $m/2$  negative genes are selected.

## OTHER APPROACHES

We conclude this chapter by presenting 2 more powerful approaches in pattern representation.

The first one is known as **LBP** (*Local Binary Pattern*) a type of visual descriptor used for classification in computer vision, which deals with the analysis and classification of images by machines.

The second one is based on **dissimilarity spaces**, a kind of *vector spaces* that can be applied to problems in many different domains.

## LOCAL BINARY PATTERNS

LBP was first described in 1994 and it has since been found to be a powerful feature for **texture classification**.

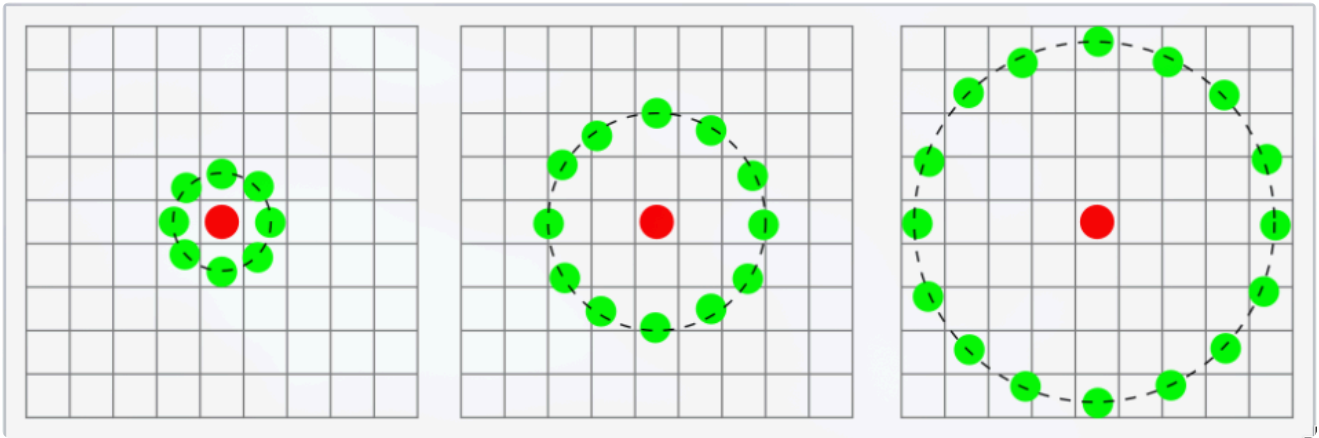
Given an *image* divided into *windows*, the LBP feature vector, in its simplest form, is created in the following manner:

### 📋 OBTAINING THE LBP VECTOR

1. **Divide** the examined window into *cells* (e.g. 16x16 pixels for *each cell*).
2. For *each pixel in a cell* **compare** the pixel to its **8 neighbours** (top-left, top-middle, top-right, etc., following the pixels **along a circle** (i.e. clockwise or counter-clockwise)).
3. Where the **centre** pixel's value is **greater** than the neighbour value **write 0**; otherwise, write 1. This gives an **8-digit binary number**, which can be thought of as a vector of 8 components, or a decimal number in  $[0, 255]$  for convenience.
4. **Compute** the histogram, **over the cell** (16x16 in the example we're considering), of the *frequency of each "number" occurring*. This histogram can be seen as a 256-dimensional feature vector.
5. **Concatenate** (*normalized*) histograms of all cells to obtain a feature vector for the *entire window*.

The LBP operator was also extended to use *neighbourhoods of different sizes*. Using a *circular neighbourhood* and bilinearly interpolating values at non-integer coordinates

allow *any radius* and *number of pixels* in the neighbourhood.



The LBP value for a neighbourhood of  $P$  points and radius  $R$  (The neighbourhood is thus denoted by  $(P, R)$ ) is given by:

$$\text{LBP}_{P,R} = \sum_{p=0}^{P-1} s(g_p - g_C) 2^p$$

Where:

$$s(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

And  $g_p$  denotes the *grayscale value* of pixel  $p$ .

A useful *extension* to the original operator is the so-called **uniform pattern**, which can be used to **reduce the length of the feature vector** and implement a simple **rotation invariant descriptor**.

This idea is motivated by the fact that **some binary patterns occur more commonly in texture images** than others.

A local binary pattern is *called uniform* if the binary pattern *contains at most two 0-1 or 1-0 transitions*. For example, 00010000 (2 transitions) is a uniform pattern, but 01010100 (6 transitions) is not.

### 🔍 WHY UNIFORM PATTERNS ARE HELPFUL

In the computation of the *LBP histogram*, the histogram has a **separate bin for every uniform pattern**, and all **non-uniform patterns are assigned to a single bin**.

Hence why it helps **reduce the dimension of the feature vector**: using uniform patterns with  $(8, R)$  neighbourhoods, the length of the feature vector for a single cell *reduces from 256 to 59* (58 bins for all the possible uniform patterns, and another bin for all the others).

It is important to note that *images carry spatial information*, therefore the following observation must be kept in mind.

### OBSERVATION

The descriptors seen above, if *calculated over the entire image*, do **not retain spatial information**.

To **enrich** the descriptor with **local information** we **divide the image into regions** (or windows, as we called them earlier) and **calculate the descriptors for each region**.

These regions may **overlap** or not, depending on the problem.

## DISSIMILARITY SPACES

Some patterns can be described by a series of numbers, where each number is the "**similarity**" between a given "**original pattern**" (a *reference*) and another one.

This way, the reference **patterns can also be complex structures**, as long as we can come up with a way to "*measure differences between them*".

Examples of such structures are graphs, in which case a *series of distances* between graphs would be the representation vector of patterns.

The precise *mathematical tool* to deal with such cases is a **dissimilarity space**:

### DISSIMILARITY SPACE

A **vector space** in which the dimensions are defined by **dissimilarity vectors** measuring **pairwise dissimilarities** *between examples and individual objects* from the so-called *representation set*  $R$ .

### NOTE

The *representation set* can be chosen as the *complete training set*, or as a *set of carefully selected or constructed prototypes*.

Hence, a *dissimilarity representation*  $D(X, R)$  is addressed as a data-dependent mapping

$$D(\cdot, R) : X \rightarrow \mathbb{R}^n$$

from an initial set of objects  $X$  to a *dissimilarity space*, equipped with the traditional *inner product* and a *metric*.

To better understand this concept, consider the following explanation.

Assume a representation set composed of  $n$  *reference objects*  $R_1, R_2, \dots, R_n$ , and assume that we have a set of objects  $O = o_1, o_2, \dots$

Then each  $o_i \in O$  can be represented as a *dissimilarity vector*  $D_i$ :

$$D_i := [d(o_i, R_1), d(o_i, R_2), \dots, d(o_i, R_n)] \in \mathbb{R}^n$$

Where  $d(o_j, R_k)$  is a **metric** that outputs the *dissimilarity* between *object*  $o_j$  and *reference*  $R_k$ .

#### NOTE

Note that:

1. The metric  $d$  *could* be as simple as outputting the *difference* between two values.
2. More precisely,  $d$  is a **pseudometric** (see explanation below).

#### METRICS AND PSEUDOMETRICS

Remember that a **metric**  $d$  over a *set*  $M$  is a *function*

$$d : M \times M \rightarrow \mathbb{R}$$

such that for any  $x, y, z \in M$ , the following hold:

1.  $d(x, y) = 0 \Leftrightarrow x = y$
2.  $d(x, y) = d(y, x)$
3.  $d(x, z) \leq d(x, y) + d(y, z)$

For a **pseudometric**, the first condition changes to:

$$d(x, x) = 0$$

That is, one may have  $d(x, y) = 0$  for distinct values  $x \neq y$ .